

Document Plagiarism Detector

OVERVIEW

A command-line utility that compares a target text document against a local database of source documents to detect localized structural similarities, bypassing simple synonym replacements or minor edits.

Category	Specification
Tech Stack	Python
Core Algorithms	Rabin-Karp Rolling Hash, Winnowing Algorithm
Target Metric	Process a 10,000-word document in under 1 second
Output	A percentage similarity score and the indices of matching blocks

FUNCTIONAL REQUIREMENTS

- **File I/O & Sanitization:** The system must read `.txt` files, convert all text to lowercase, and strip all whitespaces and punctuation.
- **K-Gram Generation:** The system must split the sanitized text into contiguous substrings of length k (a configurable parameter, usually between 5 and 15).

-
- **Hash Computation:** The system must calculate an integer hash for every k-gram using a rolling hash function to maintain $O(N)$ time complexity. The mathematical formulation should resemble:

$$H_{next} = ((H_{prev} - c_{old} \cdot b^{k-1}) \cdot b + c_{new}) \pmod{M}$$

- **Winnowing Selection:** The system must group hashes into windows of size w and select the minimum hash value from each window to form the document's "fingerprint."
- **Similarity Scoring:** The system must compute the Jaccard similarity coefficient between the fingerprints of two documents.

MILESTONES FOR COMPLETION

1. Write the parser to clean the text and generate k-grams.
2. Implement and unit-test the rolling hash function.
3. Implement the Winnowing logic and calculate the final similarity score.